

# SOFTWARE ENGINEERING II

## Course Description

Author: Eng. Liliana Marcela Olarte, M.Sc.

lmolartem@unal.edu.co

Author: Eng. Carlos Andrés Sierra, M.Sc.

casierrav@unal.edu.co

Lecturers

Computer Engineering Program

School of Engineering

Universidad Nacional de Colombia

2026-I

# Outline

- 1 Course Overview
- 2 Syllabus
- 3 Bibliography

# Outline

1 Course Overview

2 Syllabus

3 Bibliography

# Overview

In the **Software Engineering 2** course, students will explore *advanced development methodologies*, including the definition of **functional requirements**, the design of **basic architectures** such as MVC and hexagonal, and the implementation of APIs and NoSQL databases.

Additionally, a **DevOps** culture will be encouraged through continuous integration and deployment practices. Through hands-on prototypes, students will apply these concepts to develop robust and scalable software systems.

# Goals

The **objective** is to provide students with advanced training in the **fundamentals of software development**, equipping them with the *skills* needed to design, implement, and maintain modern software systems using **agile methodologies**, **effective architectures**, and **DevOps practices**.

Through *hands-on projects*, we aim to cultivate a deep understanding of modern tools and techniques in the field.

# Learning Results I

As follows, the main learning results for this course:

- **Functional Requirements:** Define, refine, and manage requirements using **user stories**, **acceptance criteria**, and **story points**.
- **Software Architectures:** Design and implement basic architectures such as **MVC** and **hexagonal architecture**.
- **Domain-Driven Design:** Apply **DDD** principles to model complex domains.
- **Design Patterns:** Apply key **design patterns** — *creational*, *structural*, and *behavioral* — to improve software quality.
- **Web Development:** Develop web applications covering **frontend**, **backend**, **Web APIs**, and **HTTP communication**.

# Learning Results II

As follows, the main learning results for this course:

- **APIs and HTTP:** Design and document **REST APIs** following best practices.
- **Software Testing:** Implement **integration testing** and **test-driven development (TDD)** strategies using tools such as Pytest, Cucumber, and Apache JMeter.
- **DevOps and CI/CD:** Explain the **DevOps philosophy** and apply **CI/CD** practices using tools like LocalStack, GitHub Actions, Docker, and Kubernetes.

# Pre-Requisites

This is a basic course, so you must have some knowledge in:

- **Programming** in **Java**, **Python**, or **C++**.
- **Object-Oriented Programming** concepts like **classes**, **objects**, **inheritance**, and **polymorphism**.
- **Git** **basic usage**, and **GitHub** basic usage.
- Use of **IDEs** like **VS Code**, Eclipse, or PyCharm.
- Basic knowledge of **databases** and **SQL**.



# Outline

- 1 Course Overview
- 2 Syllabus
- 3 Bibliography

# Syllabus I

| Time       | Topic                                                                                                                             |
|------------|-----------------------------------------------------------------------------------------------------------------------------------|
| Week 1     | Agile Fundamentals                                                                                                                |
| Week 2-3   | Refining and Prototyping: <i>User stories, acceptance criteria (Given-When-Then), Story Mapping, Estimation, Fast Prototyping</i> |
| Week 4     | Domain-Driven Design: <i>Hexagonal Architectures</i>                                                                              |
| Week 5-6   | SOLID principles: <i>User Stories Relation, Good Practices, Quality Attributes</i>                                                |
| Week 7-8   | Patterns Design I: <i>Creational</i>                                                                                              |
| Week 8-9   | Patterns Design II: <i>Structural</i>                                                                                             |
| Week 10-11 | Patterns Design III: <i>Behavioral</i>                                                                                            |

# Syllabus II

| Time       | Topic                                                                               |
|------------|-------------------------------------------------------------------------------------|
| Week 11-12 | Web Applications: <i>Frontend, Backend, Web API, HTTP communication</i>             |
| Week 13    | Test Engineering: <i>Test-Driven Design (TDD), Pytest, Cucumber, Apache JMeter</i>  |
| Week 14    | DevOps: <i>LocalStack, GitHub Actions (CI), Docker, Kubernetes, Cloud Computing</i> |

# Grades Percentages

| Item                     | Percentage |
|--------------------------|------------|
| Quizzes                  | 10%        |
| Final Test               | 20%        |
| Laboratories & Workshops | 30%        |
| Course Project           | 40%        |

# Code of Conduct

- **Always** be **respectful** to your **classmates** and to me. You must be **kind** to everyone inside (*and outside*) the classroom.
- There is **no** best **programming language**, **tool**, or **technology**. There are only **better** or **worse** solutions.
- You must be **honest** with your work. If you **don't know something**, just **ask** us. We are **glad** to help you.
- You must be **responsible** with your work. If you don't submit **on time**, please **don't complain**.
- You must not be **disruptive** or **negatively affect** the **classroom environment**.

# Outline

- 1 Course Overview
- 2 Syllabus
- 3 Bibliography

# Bibliography

Recommended bibliography:

- **Object-Oriented Analysis and Design**, by **Grady Booch**.
- **Design Patterns: Elements of Reusable Object-Oriented Software**, by **Erich Gamma, Richard Helm, Ralph Johnson, & John Vlissides**.
- **Clean Code: A Handbook of Agile Software Craftsmanship**, by **Robert C. Martin**.
- **The DevOps Handbook: How to Create World-Class Agility, Reliability, and Security in Technology Organizations**, by **Gene Kim, Jez Humble, Patrick Debois, & John Willis**.
- **Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation**, by **Jez Humble & David Farley**.

# Outline

- 1 Course Overview
- 2 Syllabus
- 3 Bibliography



**Thanks!**

**Questions?**